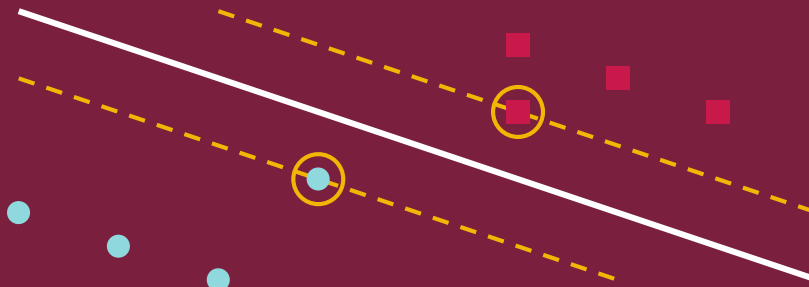


ADIMEN ARTIFIZIALA • IKASKUNTZA AUTOMATIKOA

Sostengu Bektoreen Makina

(Support Vector Machine — SVM)

Hiperplanoa, marjina eta kernel-trikuia — diagramekin azalduta



Zer da Support Vector Machine?

Definizioa

Ikaskuntza automatikoko algoritmo gainbegiratua da: klaseak ahalik eta ondoen bereizten dituen hiperplano optimoa bilatzen du, marjina maximizatuz.



Eskalatzea beharrezkoa

Distantzian oinarritua denez, aldagaiak eskalatu behar dira.



Kanpoko balioekiko sendoa

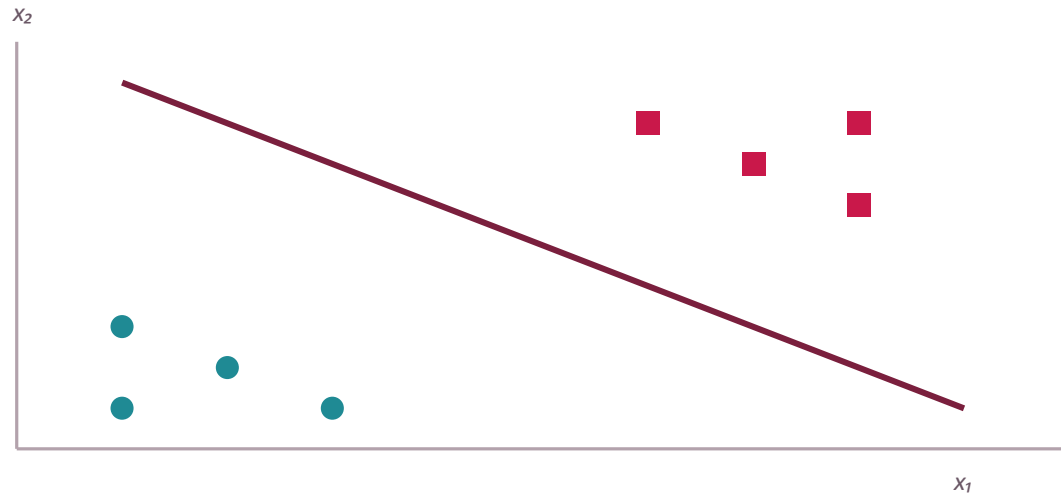
Ez da oso sentikorra outlierren aurrean.



Parametroekiko sentikorra

Datu-multzo desorekatuetan ez du oso ondo funtzionatzen.

BI KLASEEN ARTEKO BANAKETA



Kontuan izan: Datu-multzo ertainetarako oso eraginkorra da, ezaugarri esanahi antzekoa dutenean bereziki.

Oinarrizko Kontzeptu Nagusiak

H Hiperplanoa

Klase ezberdinak bereizten dituen gainazala.

M Marjina

Hiperplanoaren eta bektore sostengatzaileen arteko distantzia.

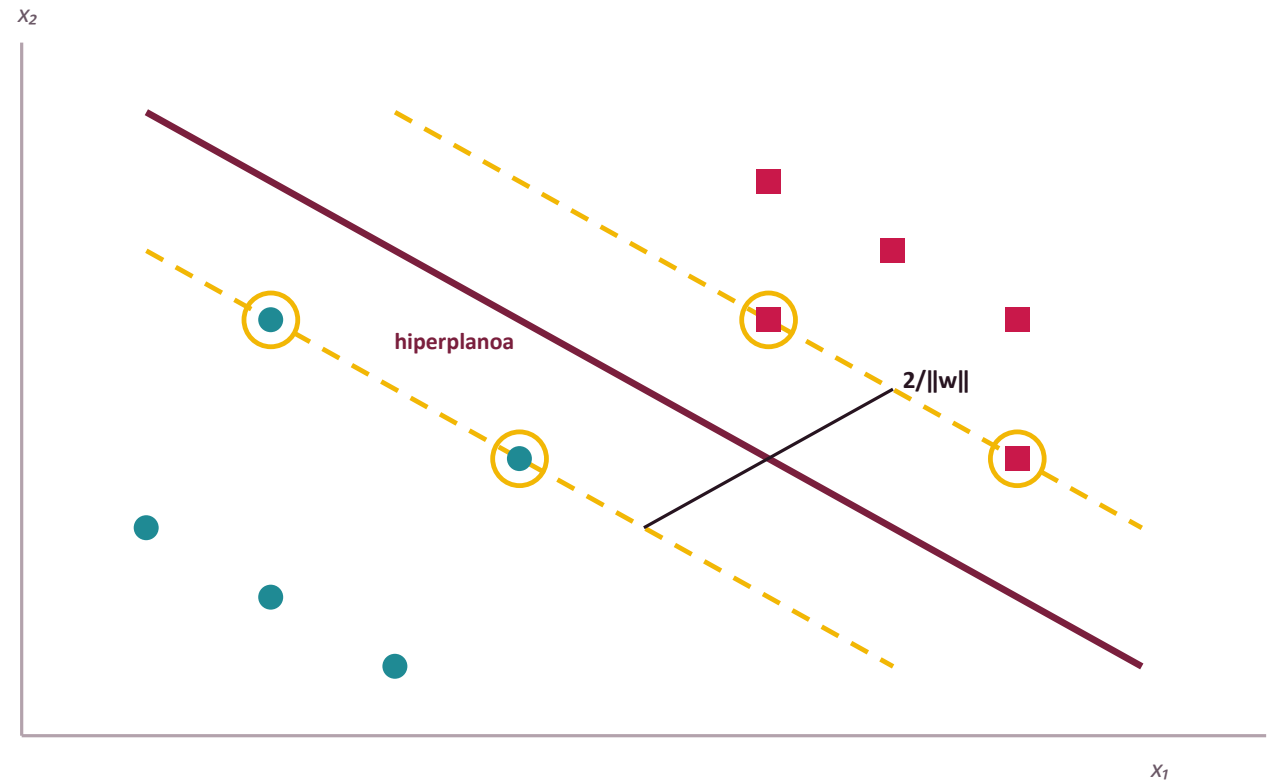
SV Bektore Sostengatzaileak

Hiperplanotik hurbilen dauden puntuak.

S Marjina Maltza / Gogorra

Maltza errorea onartzen du; gogorra ez du erroerik onartzen.

HIPERPLANOA, MARJINA ETA BEKTORE SOSTENGATZAILEAK



Marjina Gogorra (Hard Margin)

Helburua

Datu-multzoko klase guztiak akatsik gabe bereizten dituen hiperplano optimoa aurkitzea, marjina ahalik eta handiena izanik.

Ekuaizioak

$$wx + b \geq 1 \rightarrow 1 \text{ klasea}$$

$$wx + b = 0 \rightarrow \text{hiperplanoa}$$

$$wx + b \leq -1 \rightarrow -1 \text{ klasea}$$

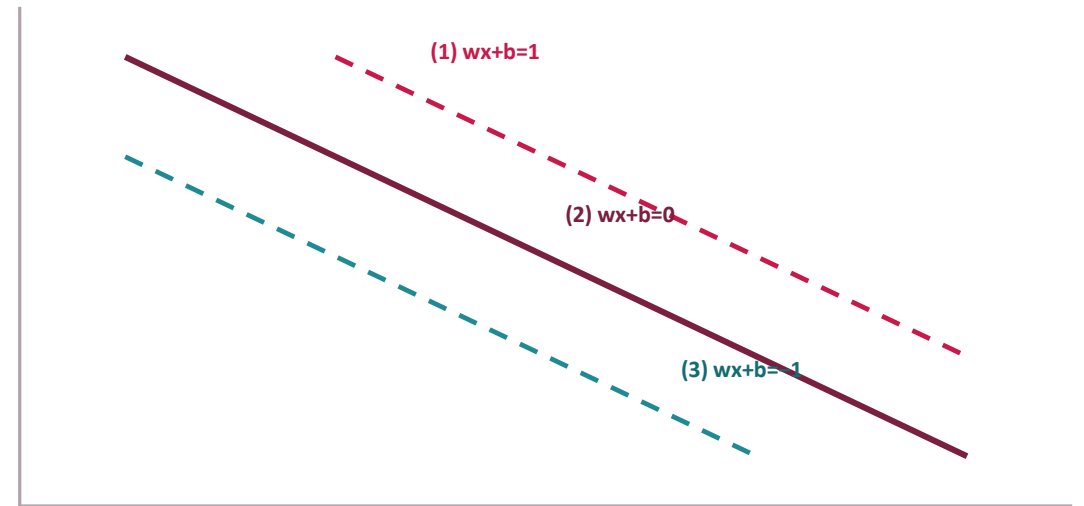


Marjinaren zabalera

$$d = 2 / \|w\|$$

$\|w\|$ minimizatzea = marjina maximizatzea.

HIRU LERRO PARALELOAK



Marjina Margua (Soft Margin) eta C Parametroa

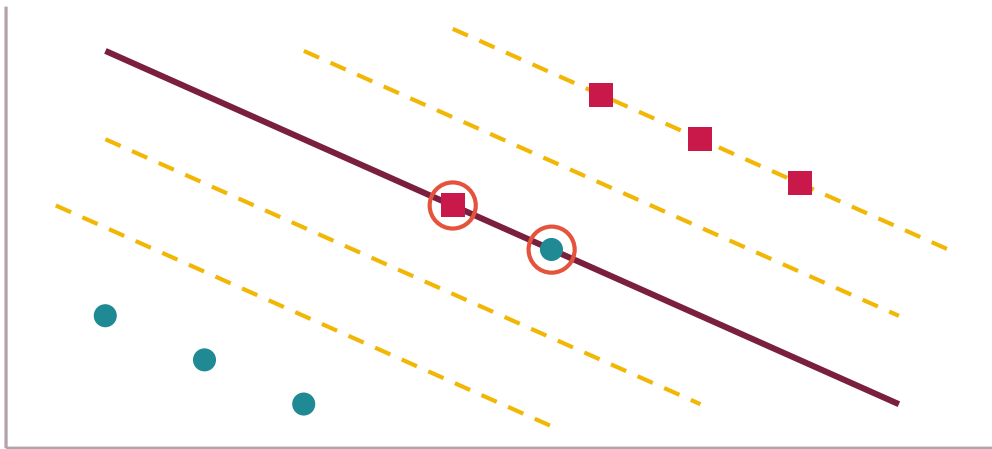
Optimizazio-helburua

$$\min \quad (1/2) \|w\|^2 + C \cdot \sum \xi_i$$

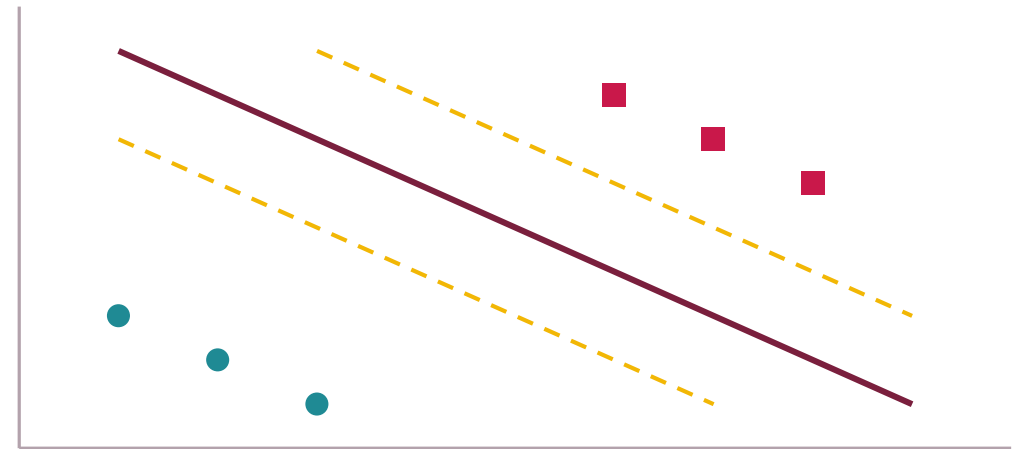
$$\text{non } y_i (w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

ξ_i (slack aldagaiak): puntu bakoitza bere marjina zuzenetik zenbat aldentzen den adierazten dute.

C txikia $\rightarrow$ marjina zabala, errore gehiago



C handia $\rightarrow$ marjina estua, errore gutxiago



C parametroa: Marjina maximizatzearen eta gaizki-sailkapena minimizatzearen arteko oreka kontrolatzen du.

SVM Kernel-a (Kernel Trick-a)

Idea nagusia

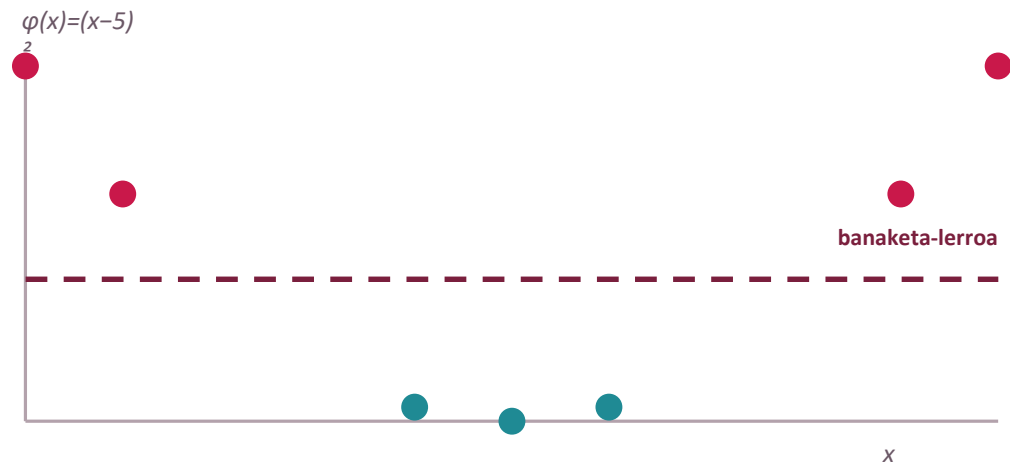
Datuak linealki bereizi ezin direnean, kernel-funtzio batek dimentsio baxuko espaziotik dimentsio altuagoko espaziora eramaten ditu, non linealki bereizgarriak diren.

JATORRIZKO ESPAZIOA (1D) — EZ BEREIZGARRIA



Klase gorria bi bazterretan eta urdina erdian: lerro batek ezin ditu bereizi.

ESPAZIO TRANSFORMATUA (2D) — BEREIZGARRIA



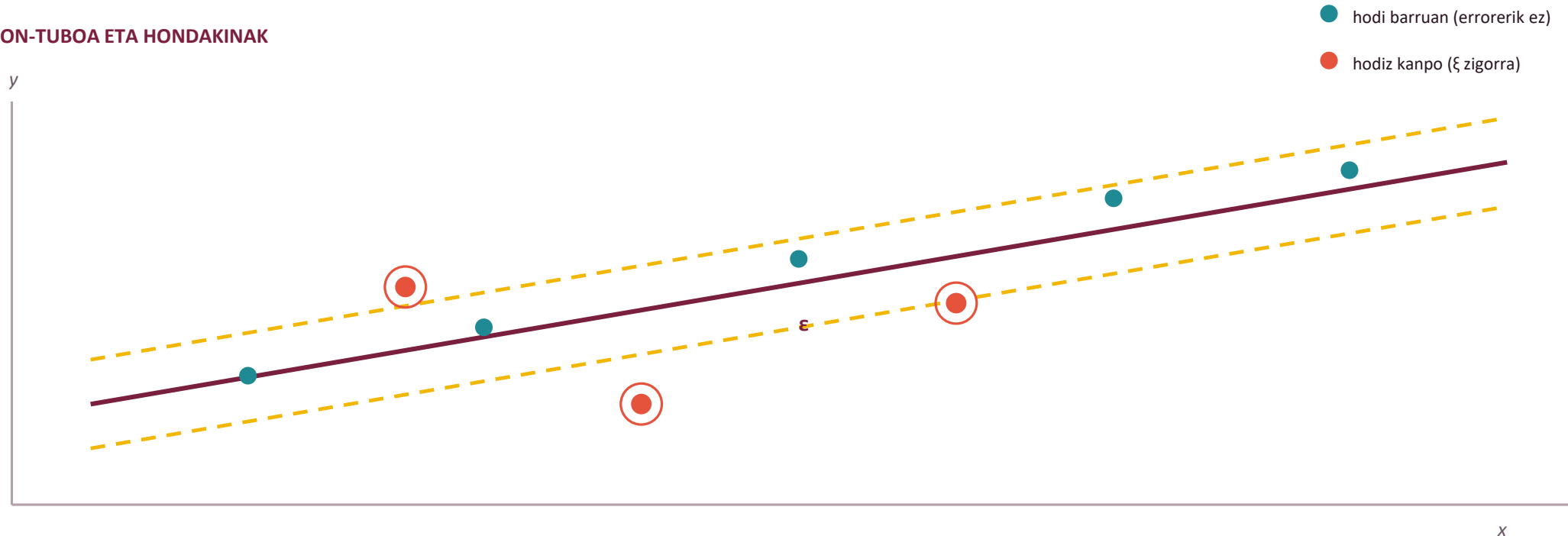
Motak: Polinomiozko kernel-a, Gaussiar RBF kernel-a eta Sigmoid kernel-a dira ohikoenak.

SVR: Erregresiorako Zabalerak-hodia (ϵ -tube)

Idea nagusia

SVR-k datuei ondoen egokitzen zaien hiperplanoa bilatzen du. ϵ (epsilon) parametroak zehazten du zenbaterainoko errorea onartzen den zigortu gabe.

EPSILON-TUBOA ETA HONDAKINAK



Abantailak eta Desabantailak



Abantailak

- **Eraginkorra dimentsio altuko espazioetan**
 - **Gainegokitzearen aurrean sendoa**
- Marjina maximizatzeak orokortze hobea dakar.
- **Eskalagarritasuna kernel-aren aukeraketarekin**



Desabantailak

- **Konputazio-kostu handia**
- Denbora-konplexutasun koadratikoa datu-multzo handietan.
- **Memoria-eskakizun handia**
 - **Funtsean sailkatzaile bitarra**
- Klase anitzetarako one-vs-all / one-vs-one behar da.
- **Interpretatzeko zaila eta parametroekiko sentikorra**
 - **Ez oso egokia datu-multzo handi eta zatatzuetarako**

Python Inplementazioa



Sailkapena (SVC)

```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X = np.array([[-1, -1], [-2, -1], [1, 1], [2, 1]])
y = np.array([1, 1, 2, 2])
clf = make_pipeline(StandardScaler(), SVC(gamma='auto'))
clf.fit(X, y)
clf.score(X, y)
```



Erregresioa (SVR)

```
from sklearn.svm import SVR
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
import numpy as np

n_samples, n_features = 10, 5
rng = np.random.RandomState(0)
y = rng.randn(n_samples)
X = rng.randn(n_samples, n_features)
regr = make_pipeline(StandardScaler(), SVR(C=1.0, epsilon=0.2))
regr.fit(X, y)
regr.score(X, y)
```

LABURPENA

SVM, urratsez urrats

1

Klaseak bereizten dituen hiperplano optimoa bilatu, marjina maximizatuz

2

Marjina gogorra (errorerik gabe) edo malgua (C bidez erregulatua) aukeratu

3

Datuak linealki bereizgarriak ez badira, kernel-trikua aplikatu (RBF, polinomikoa...)

4

Erregresiorako, ϵ -tuboaren barruko puntuak ez dira zigortzen (SVR)